INSTITUTO SUPERIOR TECNOLÓGICO PRIVADO IDAT CARRERA DE DESARROLLO DE SISTEMAS FRONT-END Y BACK-END - EEST VIRTUAL

CURSO

Desarrollo de Servicios Web

TEMA

Parcial definitivo

DOCENTE

Torres Segovia Eduar Enrique

INTEGRANTES

Jimenez Rojas, Jonathan Jose

Revilla Huapaya, Javier Alberto

Talledo Ceverino, Andy Steve

Lima - Perú

2026

API de Gestión de Citas Médicas 4
1. Introducción 4
2. Arquitectura del Sistema 4
3. Estructura detallada del proyecto 5
4. Componentes del Sistema 6
5. Endpoints de la API - Pacientes 6
6. Endpoints de la API - Médicos 7
7. Endpoints de la API - Citas 7
8. Flujo de Registro de Citas 7
9. Cálculo de Edad del Paciente 8
10. Estado de los Médicos 8
11. Sistema de Notificaciones 9
12. Interfaz de Notificación 9
13. Decisiones de diseño y control de datos en la API. 9
13.1. Uso de objetos anónimos para estructurar respuestas 9
13.2. Implementación de campos de auditoría en la entidad Cita 10
14. Validaciones del Sistema 11
15. Pruebas de los Endpoints 11
15.1. Prueba GET Cita /api/citas 11
15.2. Prueba POST registro de Cita /api/citas 12
15.3. Prueba GET búsqueda por ID Citas /api/citas/1 13
15.4. Prueba PUT actualizar Cita /api/citas/1 14
15.5. Prueba DELETE eliminar Cita /api/citas/1 15
15.6. Validaciones POST Cita (Required / StringLength) 16
15.7. Prueba GET Pacientes /api/pacientes 19
15.8. Prueba POST registro de Paciente /api/pacientes 19
15.9. Prueba GET búsqueda por ID Paciente /api/pacientes/6 20
15.10. Prueba PUT actualizar Paciente /api/pacientes/6 20
15.11. Prueba DELETE eliminar Paciente /api/pacientes/6 21
15.12. Validaciones POST Paciente (Required / StringLength) 22
15.13. Prueba GET Médicos /api/medicos 22
15.14. Prueba POST registro de Médico /api/medicos 23
15.15. Prueba GET búsqueda por ID Médicos /api/medicos/6 24

15.16. Prueba PUT actualizar Médico /api/medicos/6 24
15.17. Prueba DELETE eliminar Médico /api/medicos/6 25
15.18. Validaciones POST Medico (Required / StringLength) 26
16. Conclusiones 27

API de Gestión de Citas Médicas

1. Introducción

El presente proyecto consiste en el desarrollo de una API REST para la gestión de citas médicas.
La aplicación permite registrar, consultar, actualizar y eliminar información relacionada con pacientes, médicos y citas médicas.

La API fue desarrollada utilizando ASP.NET Core y el lenguaje C#. El proyecto fue realizado con fines académicos para demostrar el funcionamiento de endpoints HTTP, controladores, interfaces y lógica de aplicación.

La información se almacena temporalmente en memoria dentro de la aplicación, por lo que no se utiliza una base de datos externa.

2. Arquitectura del Sistema

La arquitectura del sistema se basa en una API REST. Los clientes pueden interactuar con la aplicación mediante solicitudes HTTP.

Estructura general:

Cliente (Postman / Aplicación)
|
v
API REST (ASP.NET Core)
|
v
Controladores
|
v
Almacenamiento en memoria

Durante el desarrollo se utilizó Postman para realizar pruebas de los endpoints.

3. Estructura detallada del proyecto

El proyecto se encuentra organizado de la siguiente manera:

Controllers/

CitaController.cs

MedicoController.cs

PacienteController.cs

Models/

Persona.cs

Paciente.cs

Medico.cs

Cita.cs

Interfaces/

INotificacion.cs

Notificaciones/

NotificacionSMS.cs

NotificacionCorreo.cs

NotificacionWhatsApp.cs

docs/

Informe_API_Citas_Medicas.pdf

postman/

CitasAPI.postman_collection.json

4. Componentes del Sistema

Controladores:

Reciben las solicitudes HTTP y ejecutan la lógica necesaria para cada operación.

Modelos:

Representan las entidades principales del sistema como Paciente, Médico y Cita.

Interfaces:

Se utilizan para definir comportamientos comunes, como el sistema de notificaciones.

Almacenamiento en memoria:

Los datos se almacenan en colecciones dentro de la aplicación mientras esta se encuentra en ejecución.

5. Endpoints de la API - Pacientes

GET /api/pacientes

Obtiene la lista de pacientes registrados.

GET /api/pacientes/{id}

Obtiene un paciente específico por su identificador.

POST /api/pacientes

Registra un nuevo paciente.

PUT /api/pacientes/{id}

Actualiza la información de un paciente existente.

DELETE /api/pacientes/{id}

Elimina un paciente del sistema.

6. Endpoints de la API - Médicos

GET /api/medicos

Obtiene la lista de médicos registrados.

GET /api/medicos/{id}

Obtiene un médico específico.

POST /api/medicos

Registra un nuevo médico.

PUT /api/medicos/{id}

Actualiza la información de un médico.

DELETE /api/medicos/{id}

Elimina un médico del sistema.

7. Endpoints de la API - Citas

GET /api/citas

Obtiene todas las citas registradas.

GET /api/citas/{id}

Obtiene una cita específica.

POST /api/citas

Registra una nueva cita médica.

PUT /api/citas/{id}

Actualiza una cita existente.

DELETE /api/citas/{id}

Elimina una cita.

8. Flujo de Registro de Citas

1) El cliente envía una solicitud POST a la API.
2) El controlador recibe la solicitud.

3) Se validan los datos recibidos.
4) Se registra la cita en la colección en memoria.
5) Se ejecuta el sistema de notificaciones.
6) La API devuelve una respuesta al cliente.

9. Cálculo de Edad del Paciente

En el sistema no se muestra directamente la fecha de nacimiento del paciente al consultar la información. En su lugar se calcula la edad automáticamente.

La edad se obtiene restando el año actual menos el año de nacimiento del paciente y verificando si ya cumplió años en el año actual.

```
public class Paciente : Persona
    public int Edad
    {
        get
        {
            int edad = DateTime.Now.Year - FechaNacimiento.Year;
            // Condicionamos a que si aún no llega su fecha de nacimiento
            // del año en curso, le reste 1 año a su edad
            if (DateTime.Now.DayOfYear < FechaNacimiento.DayOfYear)
            {
                edad--;
            }

            return edad;
        }
    }
}
```

10. Estado de los Médicos

Cada médico posee un estado que puede ser:

- Activo
- Inactivo

Esto permite controlar si un médico se encuentra se encuentra en actividad o no sin necesidad de eliminar completamente la información.

11. Sistema de Notificaciones

El sistema implementa un mecanismo de notificaciones basado en una interfaz.

Las notificaciones se ejecutan cuando se realizan operaciones importantes como:

- Registro de una cita
- Actualización de una cita
- Eliminación de una cita

12. Interfaz de Notificación

La interfaz INotificacion define el comportamiento que deben implementar los diferentes tipos de notificación.

Estructura conceptual:

```
INotificacion
   |-- NotificacionSMS
   |-- NotificacionCorreo
   |-- NotificacionWhatsApp
```

Este diseño permite que el sistema sea flexible y extensible.

13. Decisiones de diseño y control de datos en la API.

13.1. Uso de objetos anónimos para estructurar respuestas.

En los endpoints de la API se utilizaron **objetos anónimos** para estructurar las respuestas, permitiendo mostrar únicamente los campos necesarios de las entidades **Paciente** y **Médico** y evitando exponer información innecesaria o sensible. Además, esta técnica facilita organizar los datos en un formato más claro y ordenado en las respuestas JSON, mejorando el control de la información expuesta y siguiendo buenas prácticas en el diseño de servicios REST.

```
Paciente = pacienteEncontrado is null ? null : new
{
    pacienteEncontrado.IdPaciente,
    pacienteEncontrado.Nombres,
    pacienteEncontrado.ApePaterno,
    pacienteEncontrado.ApeMaterno,
    pacienteEncontrado.DNI,
    pacienteEncontrado.Celular,
    pacienteEncontrado.Correo,
    // No devolvemos Fecha de nacimiento
    pacienteEncontrado.Edad
},
```

13.2. Implementación de campos de auditoría en la entidad Cita.

En la entidad **Cita** se incorporaron los campos **FechaRegistro** y **FechaActualizacion** para llevar un control temporal de los registros. **FechaRegistro** se asigna automáticamente al crear la cita y permanece sin cambios, mientras que **FechaActualizacion** se modifica cada vez que se realiza una actualización. Estos campos forman parte de los campos de auditoría, permitiendo mejorar la trazabilidad y el seguimiento de los cambios en el sistema.

```
// Agregamos campos FechaRegistro y FechaActualizacion para control de auditoria
// Campo autogenerado y único
2 referencias
public DateTime FechaRegistro { get; set; } = DateTime.Now;

// Campo que inicializará en null, cambiará con la fecha actual
// en la que se hizo la modificación de la cita
3 referencias
public DateTime? FechaActualizacion { get; set; }
```

14. Validaciones del Sistema

La API incluye validaciones para asegurar que los datos enviados por el cliente sean correctos.

Entre las validaciones implementadas se encuentran:

- Campos obligatorios (Required / StringLength)
- Validación de datos nulos
- Validación de formatos de entrada

Estas validaciones permiten evitar el registro de información incompleta o incorrecta.

15. Pruebas de los Endpoints

Para validar el correcto funcionamiento de la API se realizaron pruebas utilizando Postman.

Las pruebas incluyen:

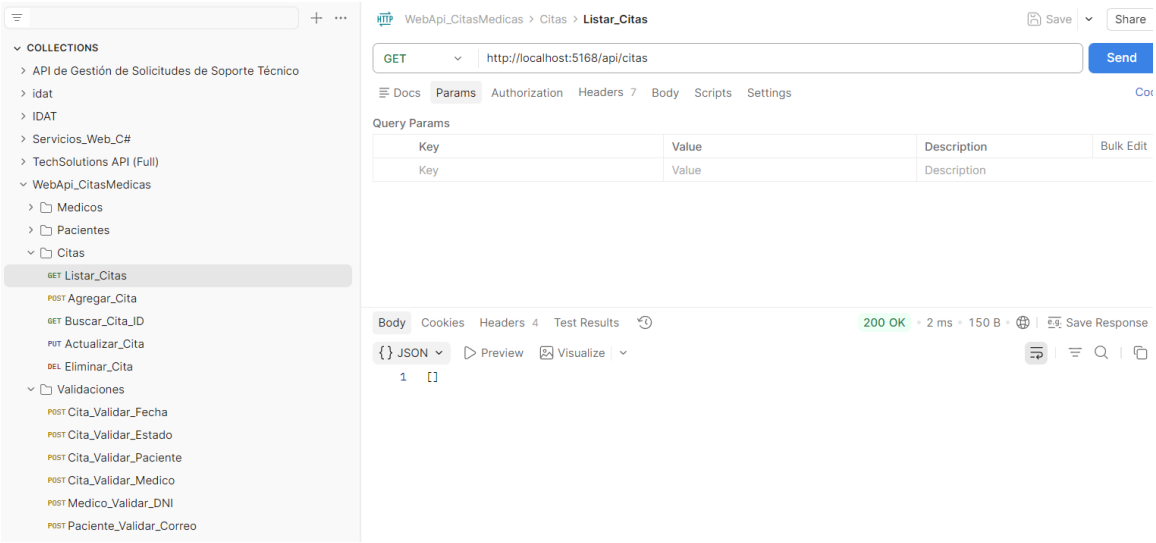
- CRUD citas
- CRUD pacientes
- CRUD médicos
- Notificaciones en consola VSC (citas)
- Validaciones

En esta sección se agregarán capturas de pantalla que muestran:

- Ejecución de cada endpoint.
- Respuestas del servidor.
- Validaciones de campos obligatorios.
- Manejo de errores cuando faltan datos requeridos.

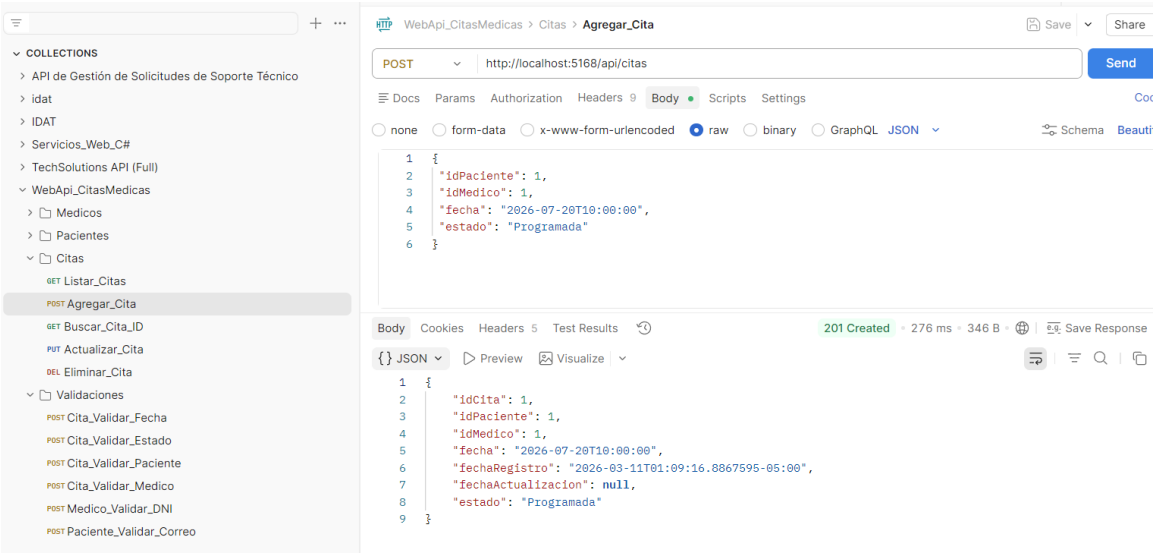
15.1. Prueba GET Cita /api/citas

Listar_Citas: En esta prueba se verifica que el endpoint devuelve correctamente la lista de citas registradas en el sistema (vacía).



15.2. Prueba POST registro de Cita /api/citas

Agregar_Cita: En esta prueba se verifica que el endpoint devuelve correctamente 201 Created, agregando la cita la lista de citas.

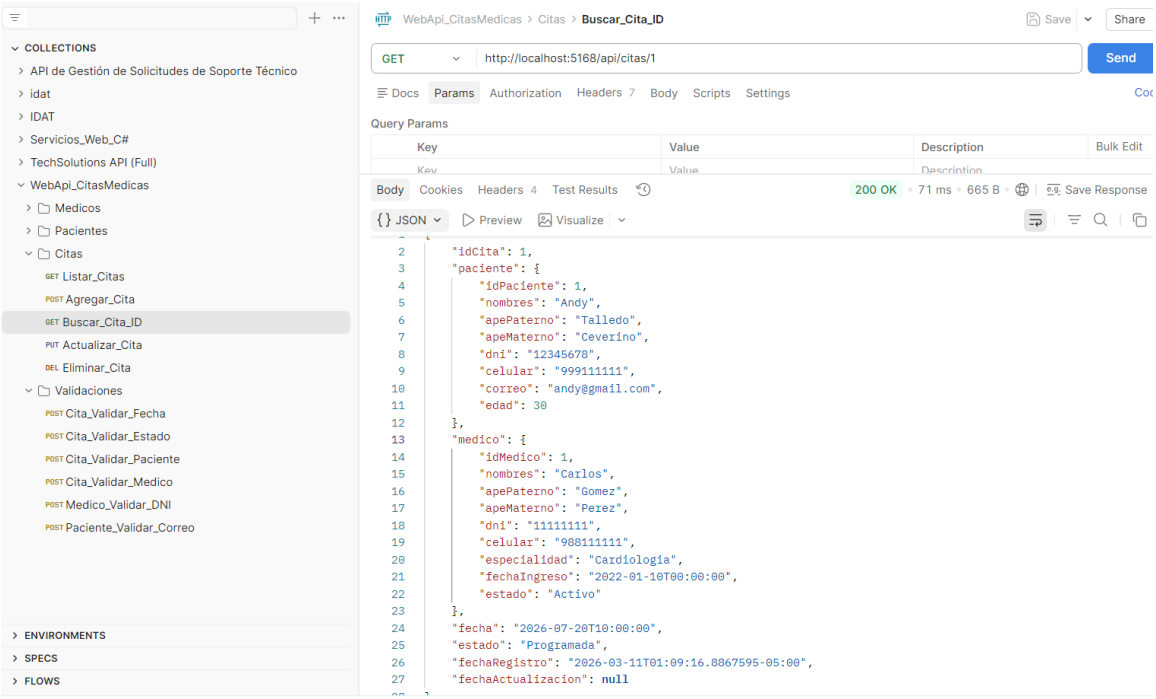


- Notificación en consola por **correo** de la cita registrada.

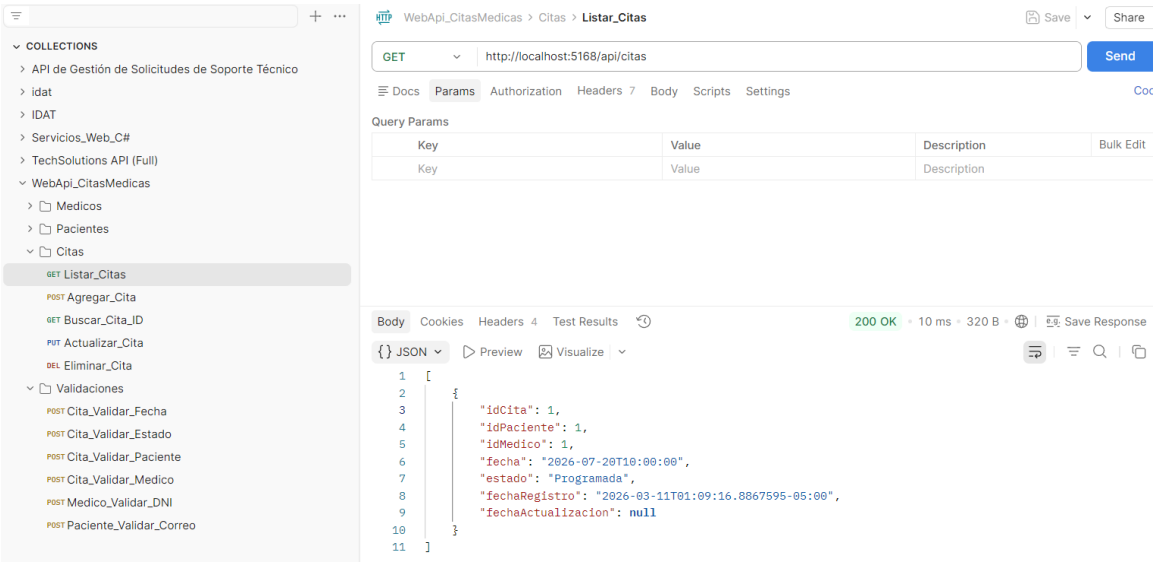
```
PROBLEMAS   SALIDA   CONSOLA DE DEPURACIÓN   TERMINAL   PUERTOS                dotnet
PS D:\Servicios_Web\parcial_grupo06> dotnet run
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\Servicios_Web\parcial_grupo06
warn: Microsoft.AspNetCore.HttpsPolicy.HttpsRedirectionMiddleware[3]
      Failed to determine the https port for redirect.
Correo enviado: Su cita médica ha sido registrada
```

15.3. Prueba GET búsqueda por ID Citas /api/citas/1

Buscar_Cita_ID: En esta prueba se verifica que el endpoint devuelve correctamente la cita buscada de acuerdo con su ID.

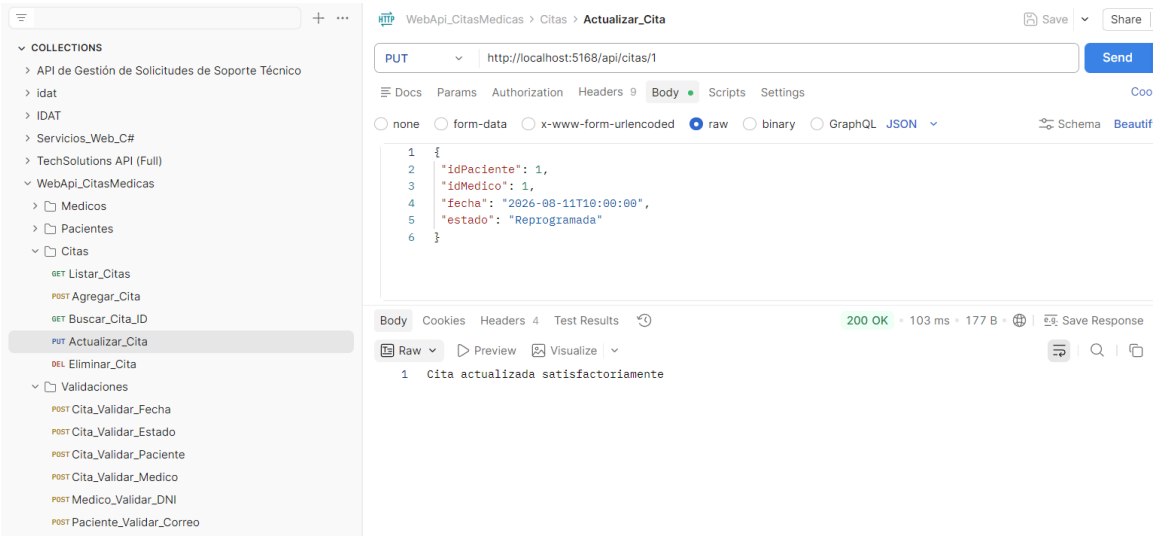


- Si buscamos por GET (lista de la colección completa), mostrará campos solo los códigos de Paciente y Médico, pudiendo obtener el detalle completo de estos en la cita "**solo al buscar por ID**" como lo muestra imagen anterior, se vuelve a mostrar GET (colección) para ver la diferencia.

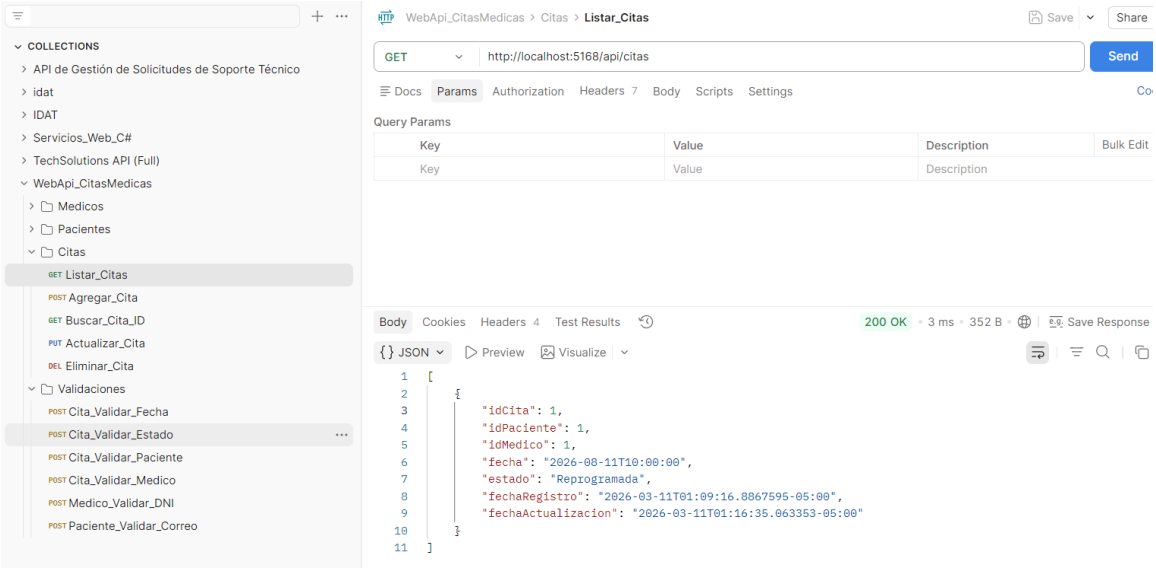


15.4. Prueba PUT actualizar Cita /api/citas/1

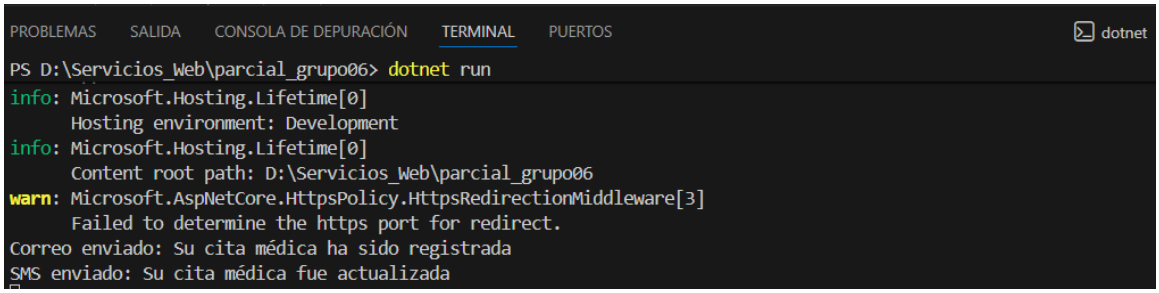
Actualizar_Cita: En esta prueba se verifica que el endpoint devuelve correctamente 200 OK, actualizando la información de la cita con ID 1.



- Fecha (de la cita) cambió de 2026-07-20 a 2026-08-11.
- Estado cambió de “Programada” a “Reprogramada”.
- Se autogeneró FechaActualización debido a la modificación.

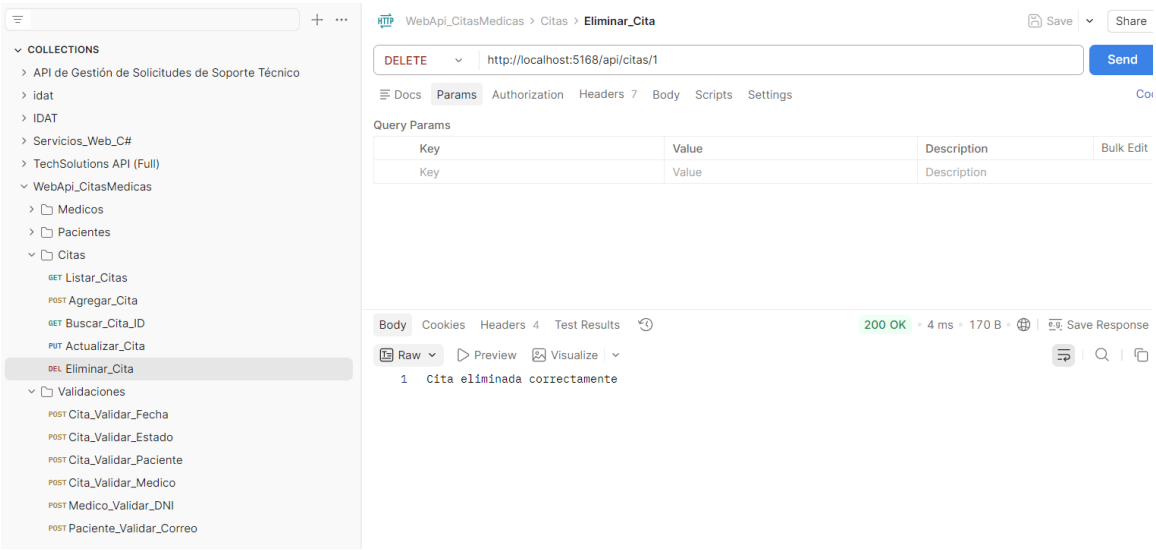


- Notificación en consola por **SMS** de la modificación cita.

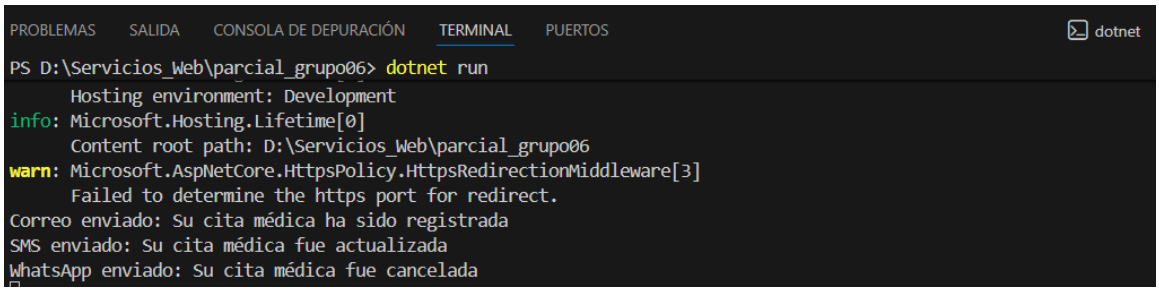


15.5. Prueba DELETE eliminar Cita /api/citas/1

Eliminar_CIta: En esta prueba se verifica que el endpoint devuelve correctamente 200 OK, eliminando la cita con ID 6.

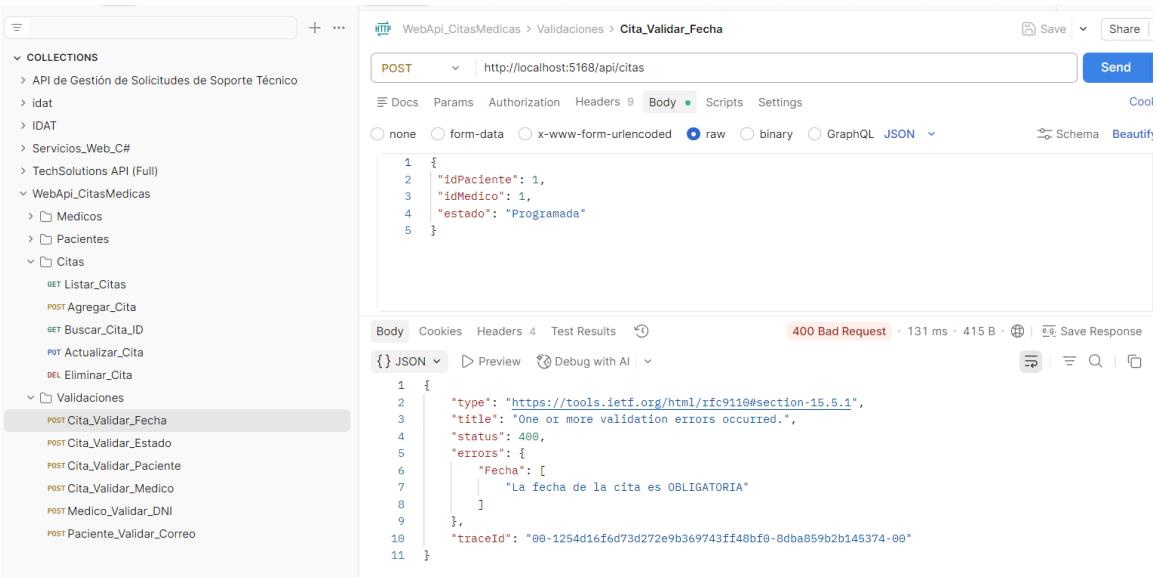


- Notificación en consola por **WhatsApp** de la cancelación de la cita.

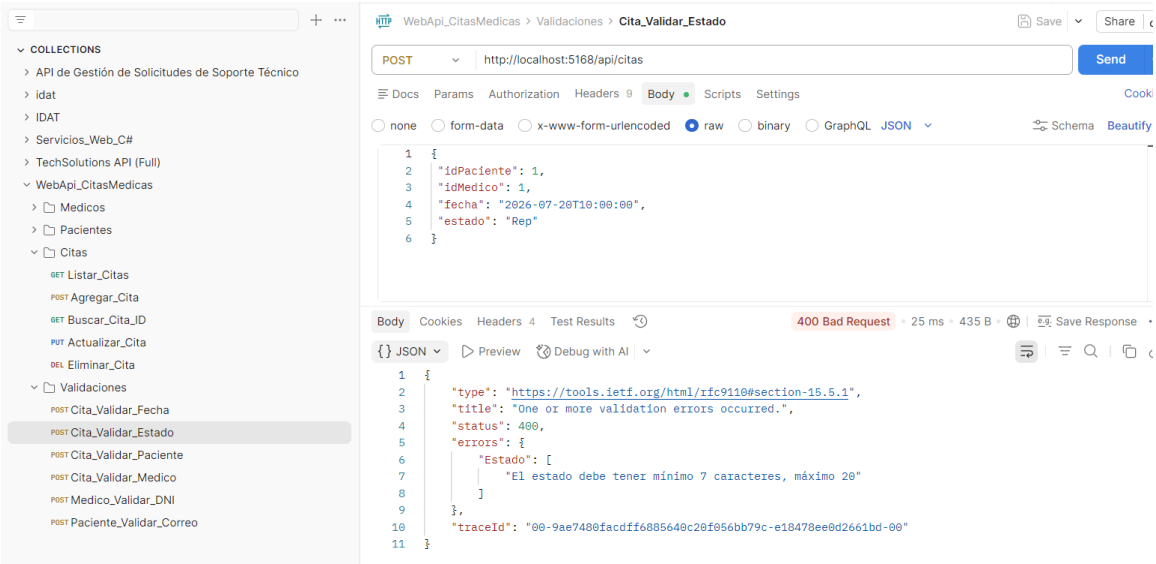


15.6. Validaciones POST Cita (Required / StringLength)

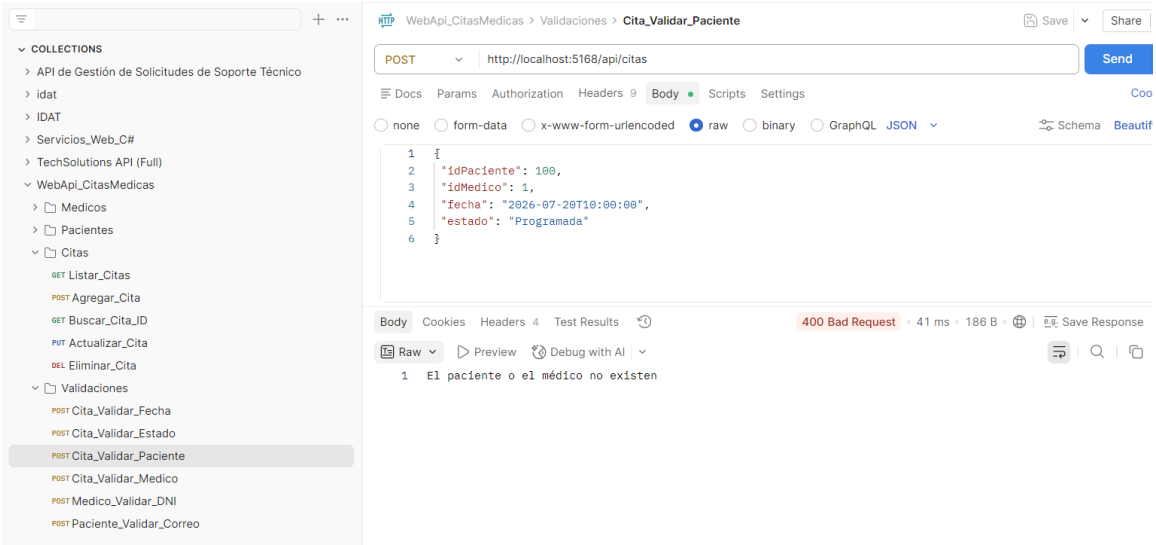
- **Cita_Validar_Fecha:** En esta prueba validamos que, si no enviamos la fecha o le colocamos null, nos indica que este dato es obligatorio.



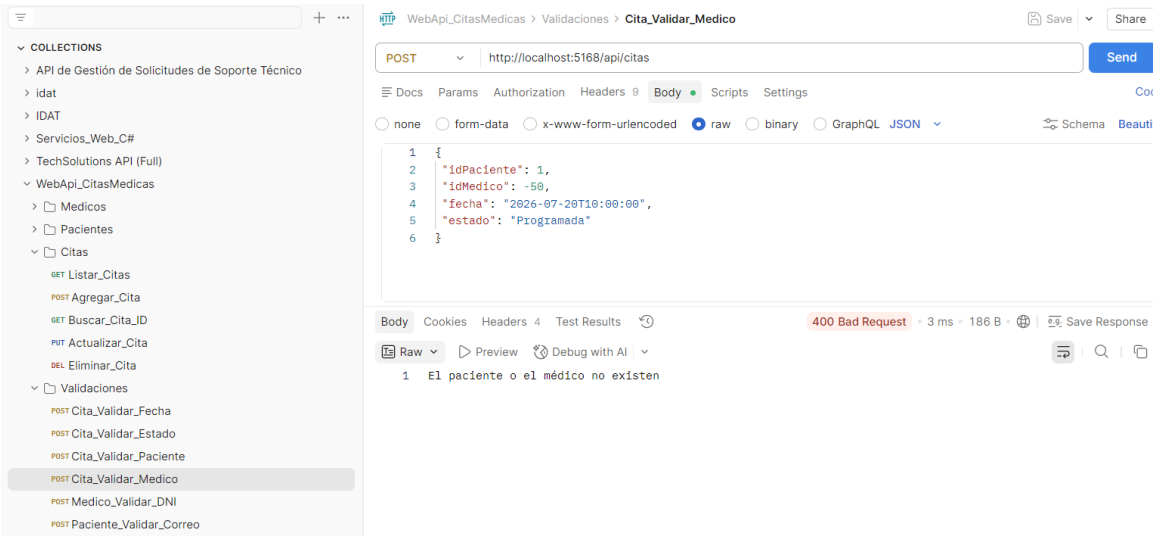
- **Cita_Validar_Estado:** En esta prueba validamos que, si enviamos el estado con una cadena menor a 7 caracteres, nos indicará que este dato es obligatorio.



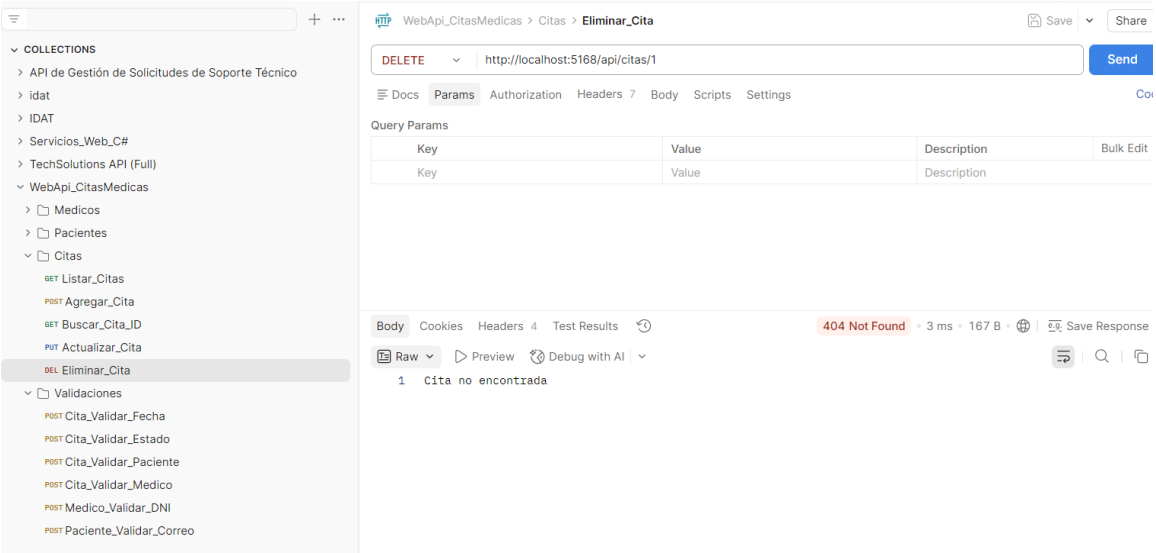
- **Cita_Validar_Paciente:** En esta prueba validamos que, si enviamos un ID de un paciente que no está registrado en sistema, o no mandamos el ID, el endpoint nos devolverá que ese paciente no existe.



- **Cita_Validar_Medico:** En esta prueba validamos que, si enviamos un ID de un médico que no está registrado en sistema, o no mandamos el ID, el endpoint nos devolverá que ese médico no existe.

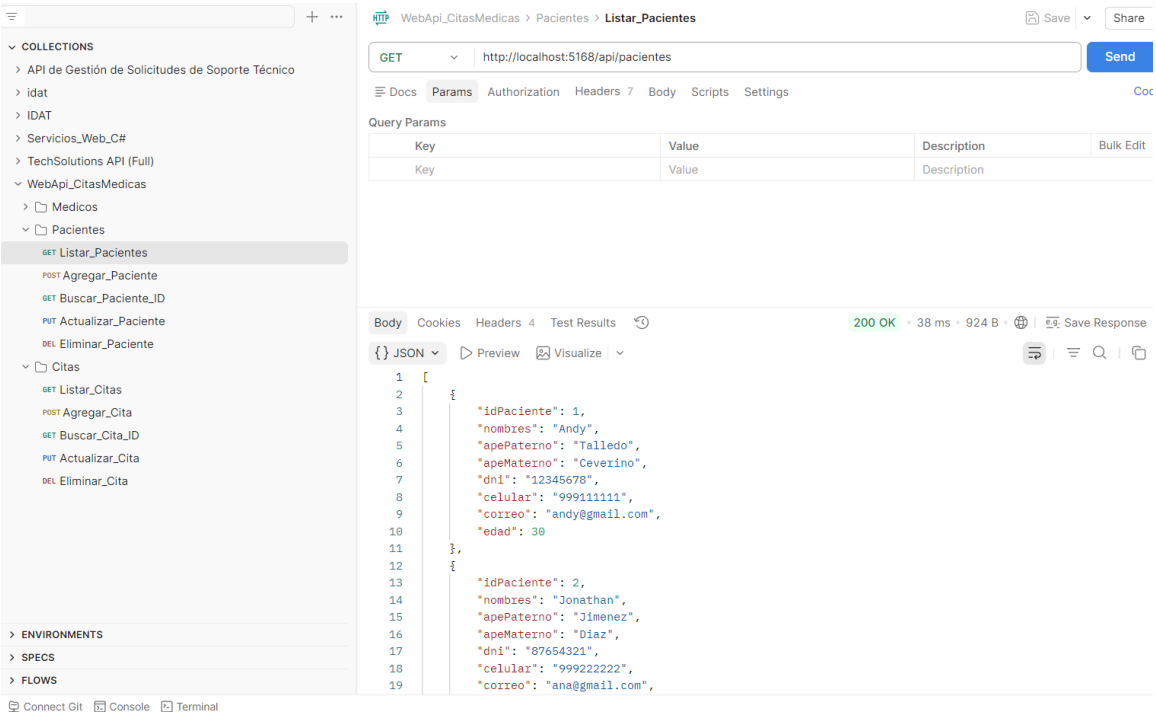


- **Eliminar_Cita:** En esta prueba validamos que, si intentamos eliminar una cita que no existe, el endpoint nos devolverá que no la encuentra.



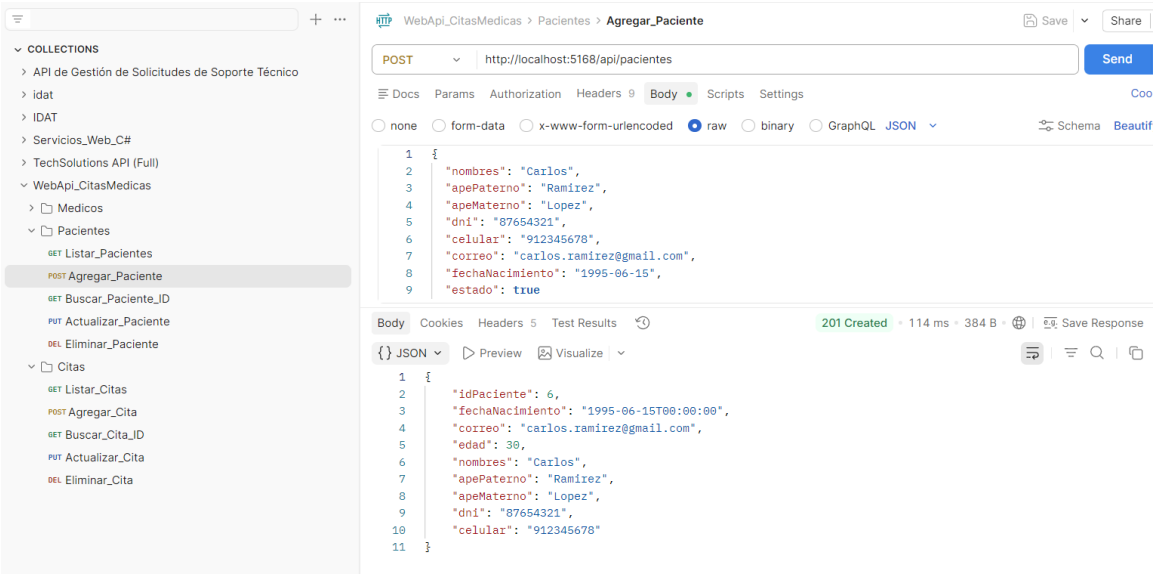
15.7. Prueba GET Pacientes /api/pacientes

Listar_Pacientes: En esta prueba se verifica que el endpoint devuelve correctamente la lista de pacientes registrados en el sistema.



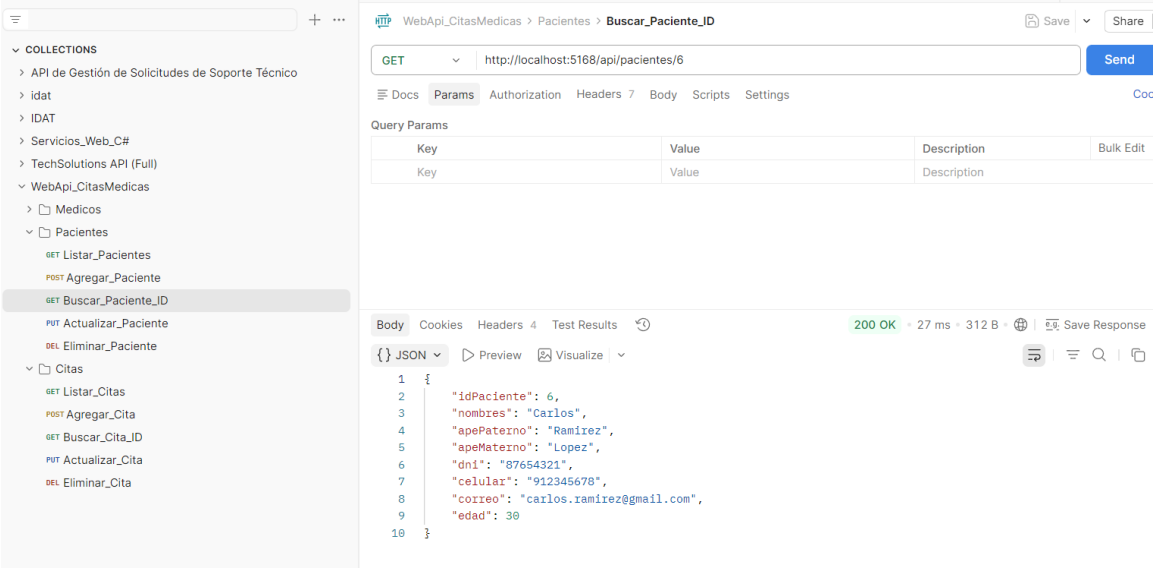
15.8. Prueba POST registro de Paciente /api/pacientes

Agregar_Paciente: En esta prueba se verifica que el endpoint devuelve correctamente 201 Created, agregando el paciente a la lista de pacientes precargados (5).



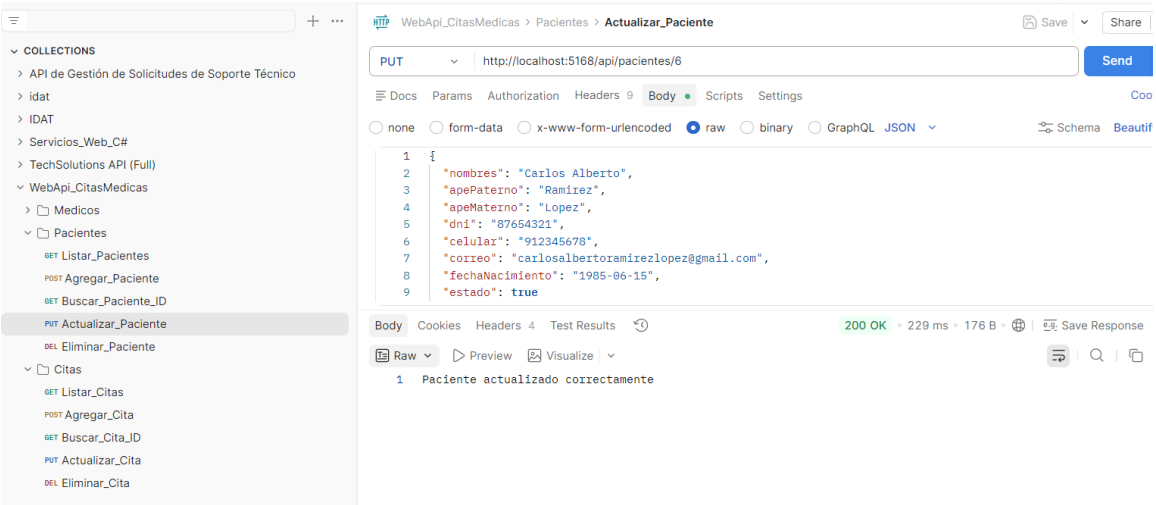
15.9. Prueba GET búsqueda por ID Paciente /api/pacientes/6

Buscar_Paciente_ID: En esta prueba se verifica que el endpoint devuelve correctamente el paciente buscado de acuerdo con su ID.

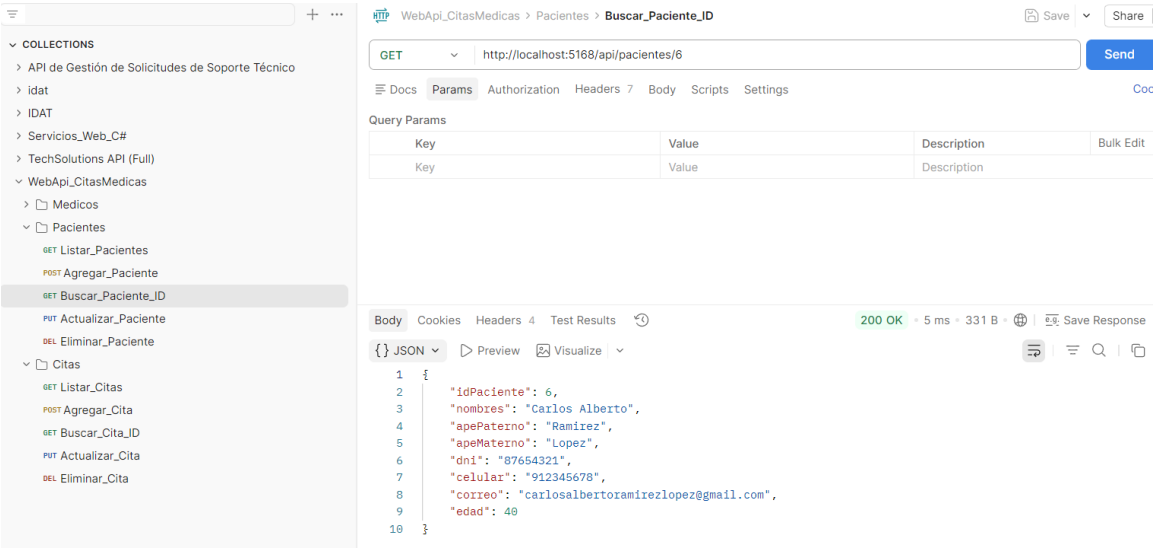


15.10. Prueba PUT actualizar Paciente /api/pacientes/6

Actualizar_Paciente: En esta prueba se verifica que el endpoint devuelve correctamente 200 OK, actualizando la información del paciente con ID 6.

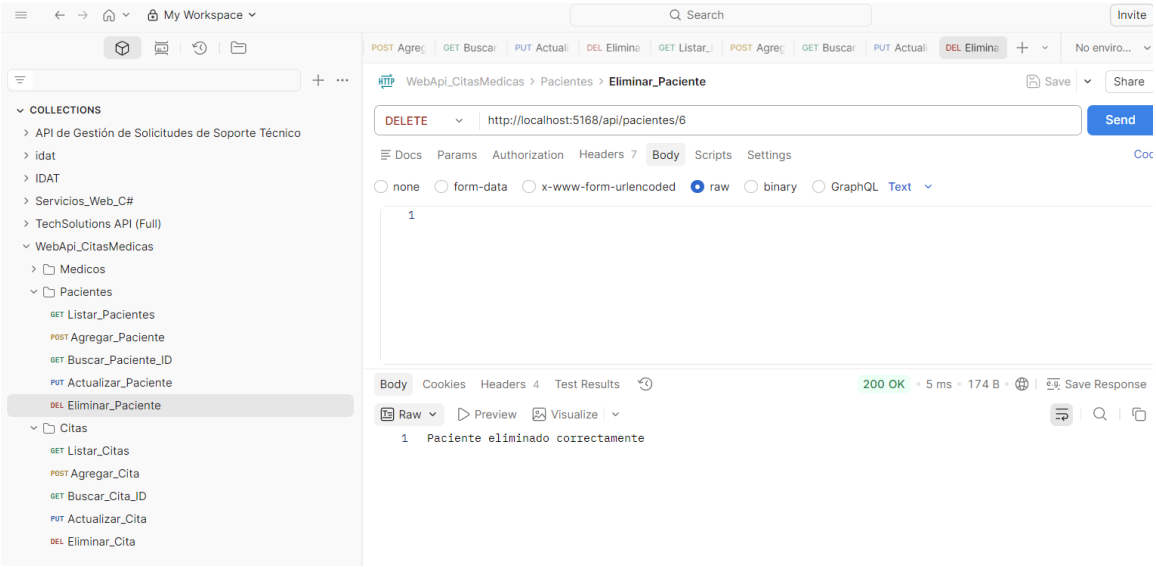


- Correo cambió de carlos.ramirez@gmail.com carlosalbertoramirezlopez@gmail.com
- Fecha de nacimiento cambió de 1995-06-15 a 1985-06-15 (Actualizando edad de 30 a 40)



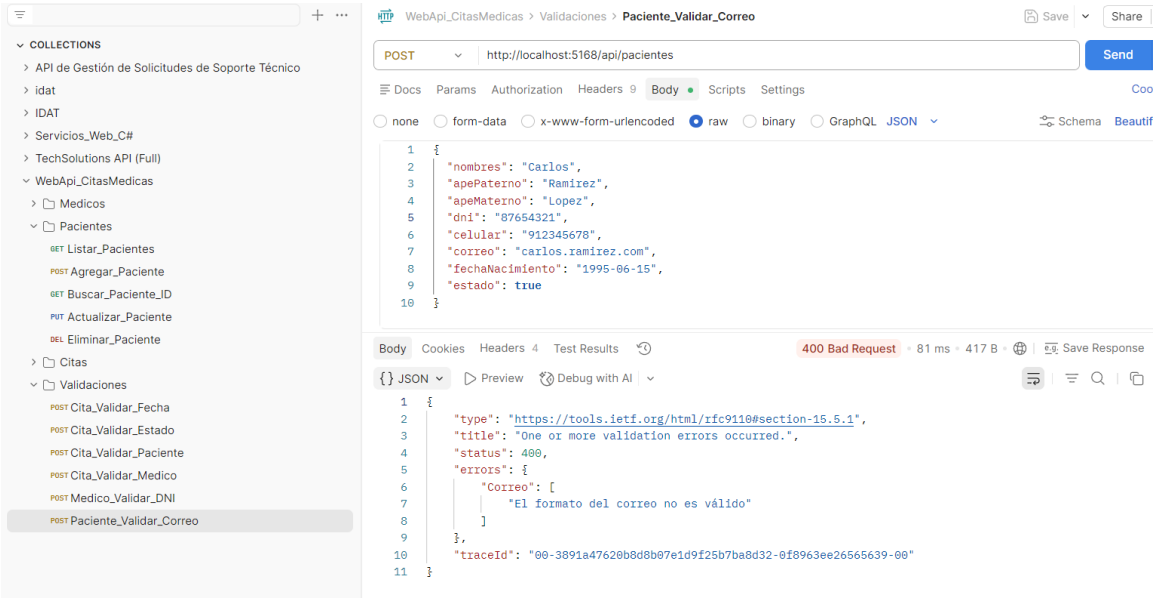
15.11. Prueba DELETE eliminar Paciente /api/pacientes/6

Eliminar_Paciente: En esta prueba se verifica que el endpoint devuelve correctamente 200 OK, eliminando el paciente con ID 6.



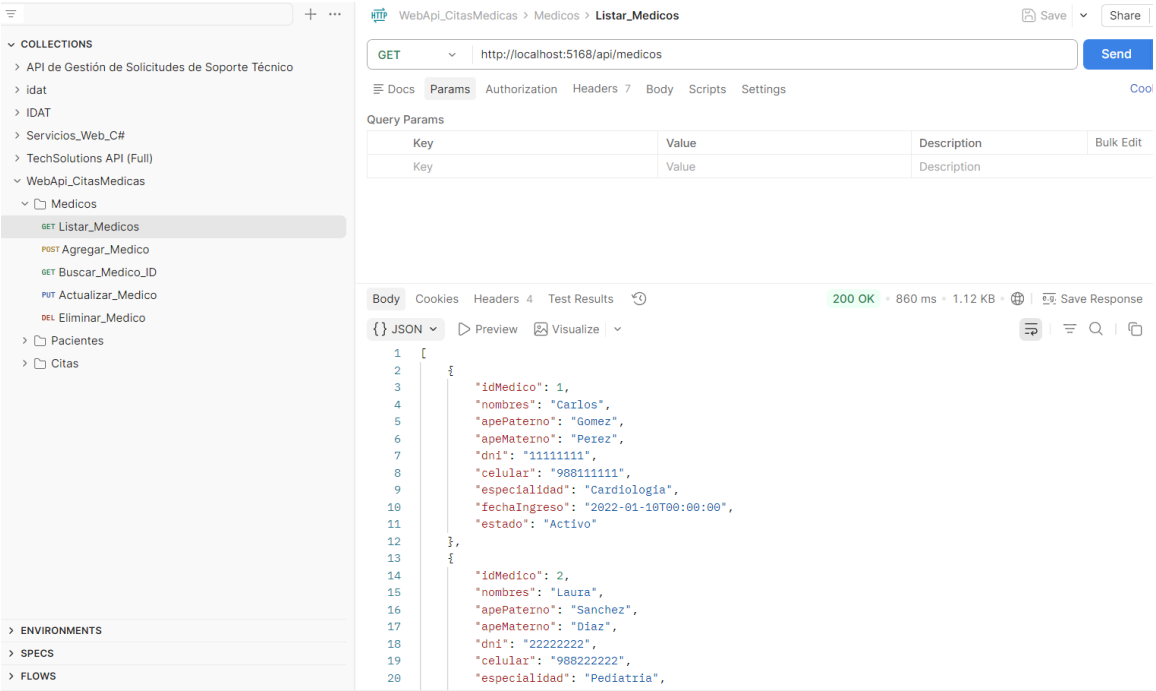
15.12. Validaciones POST Paciente (Required / StringLength)

- **Paciente_Validar_Correo:** En esta prueba validamos que, si enviamos un correo electrónico con un formato no válido, el endpoint nos devolverá el mensaje informativo correspondiente.



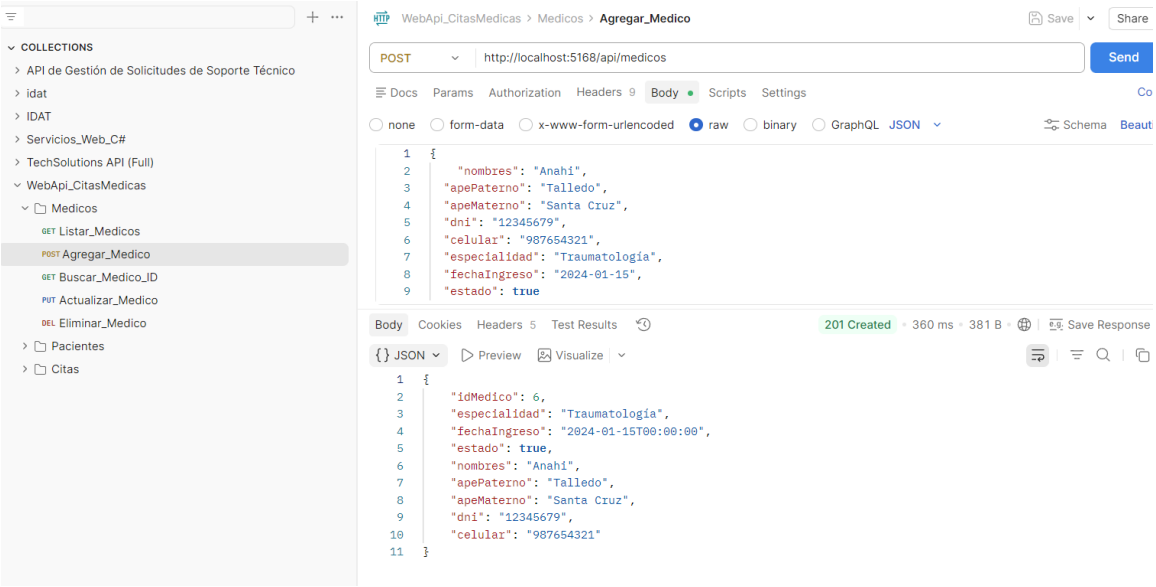
15.13. Prueba GET Médicos /api/medicos

Listar_Medicos: En esta prueba se verifica que el endpoint devuelve correctamente la lista de médicos registrados en el sistema.



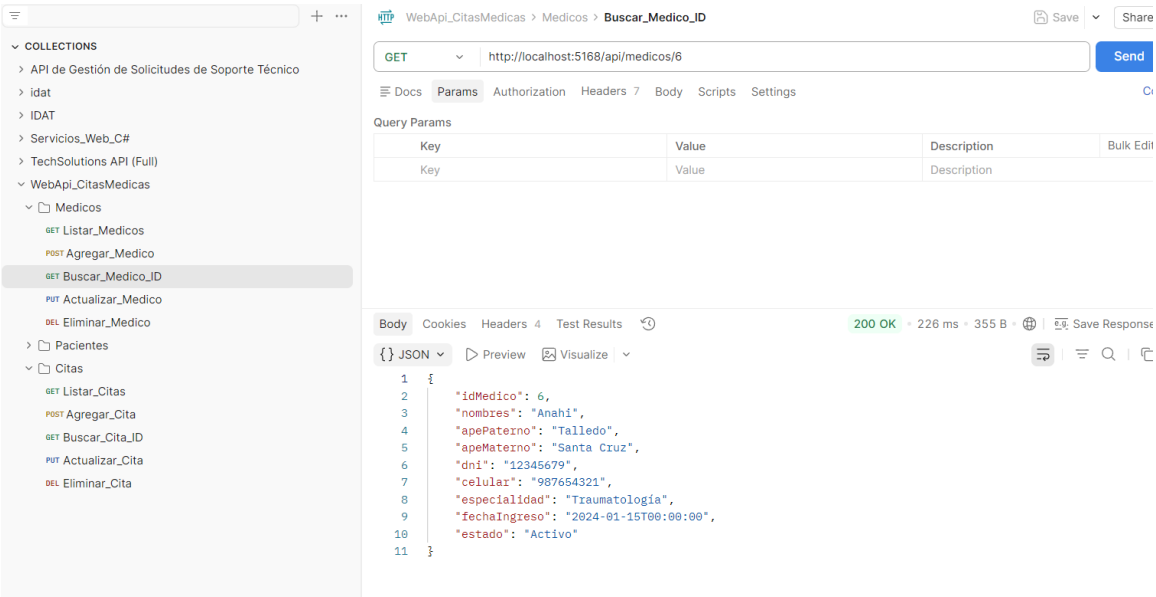
15.14. Prueba POST registro de Médico /api/medicos

Agregar_Medico: En esta prueba se verifica que el endpoint devuelve correctamente 201 Created, agregando el médico a la lista de médicos precargados (5).



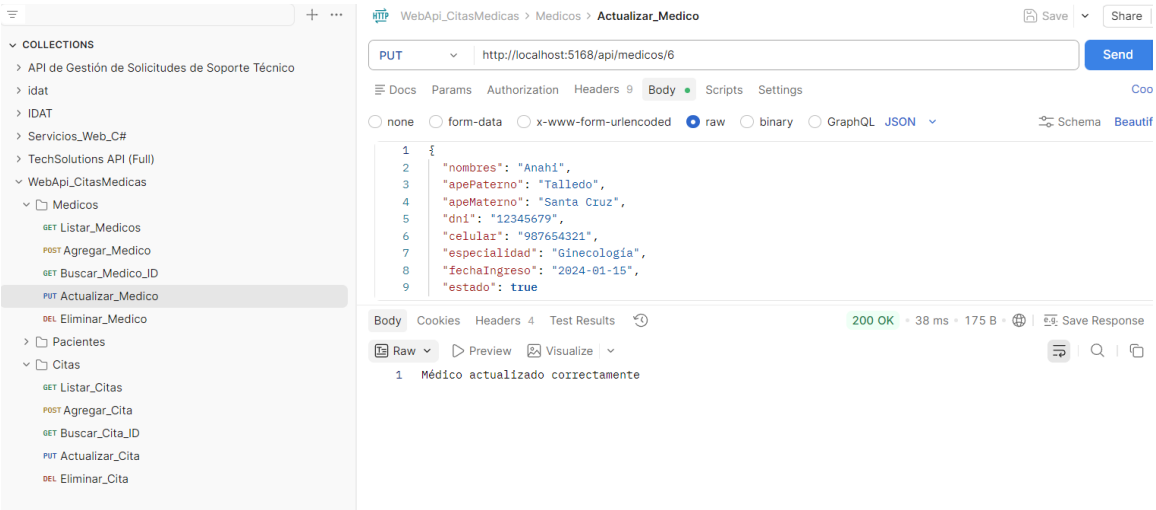
15.15. Prueba GET búsqueda por ID Médicos /api/medicos/6

Buscar_Medico_ID: En esta prueba se verifica que el endpoint devuelve correctamente el médico buscado de acuerdo a su ID.

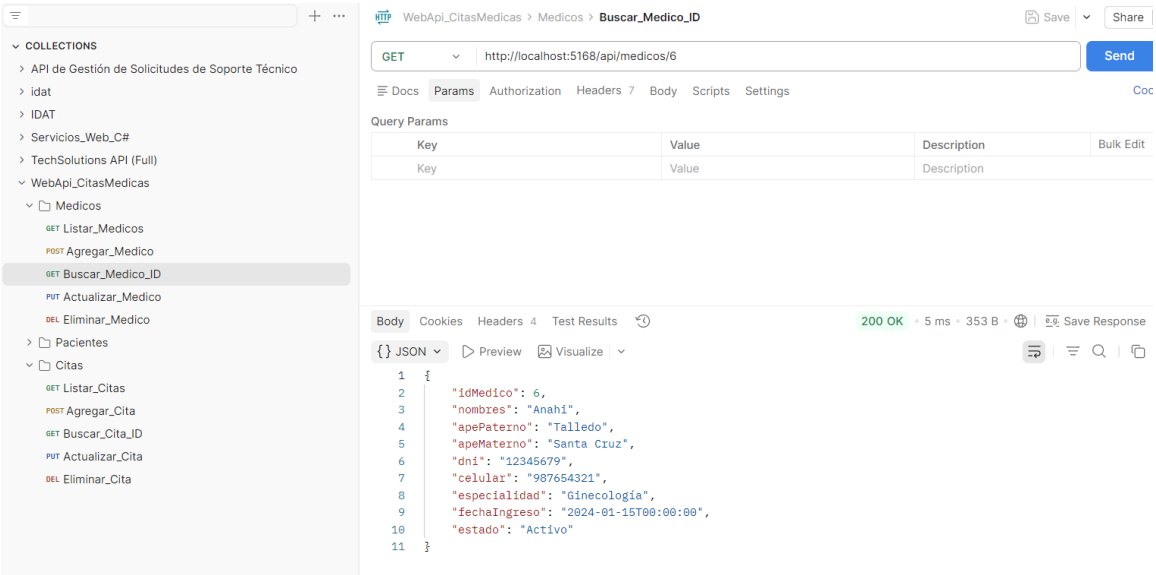


15.16. Prueba PUT actualizar Médico /api/medicos/6

Actualizar_Medico: En esta prueba se verifica que el endpoint devuelve correctamente 200 OK, actualizando la información del médico con ID 6.

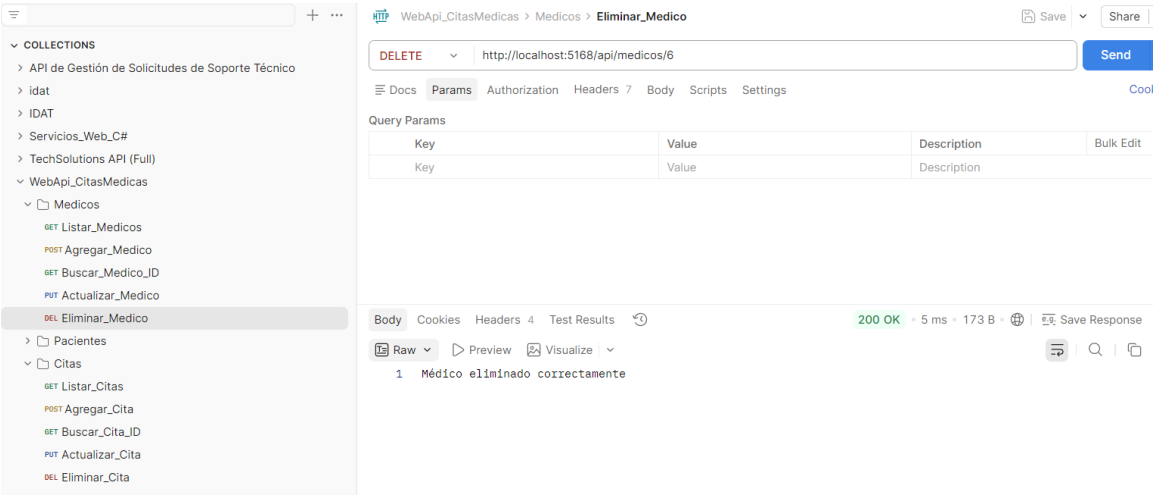


- Especialidad de traumatología a ginecología.



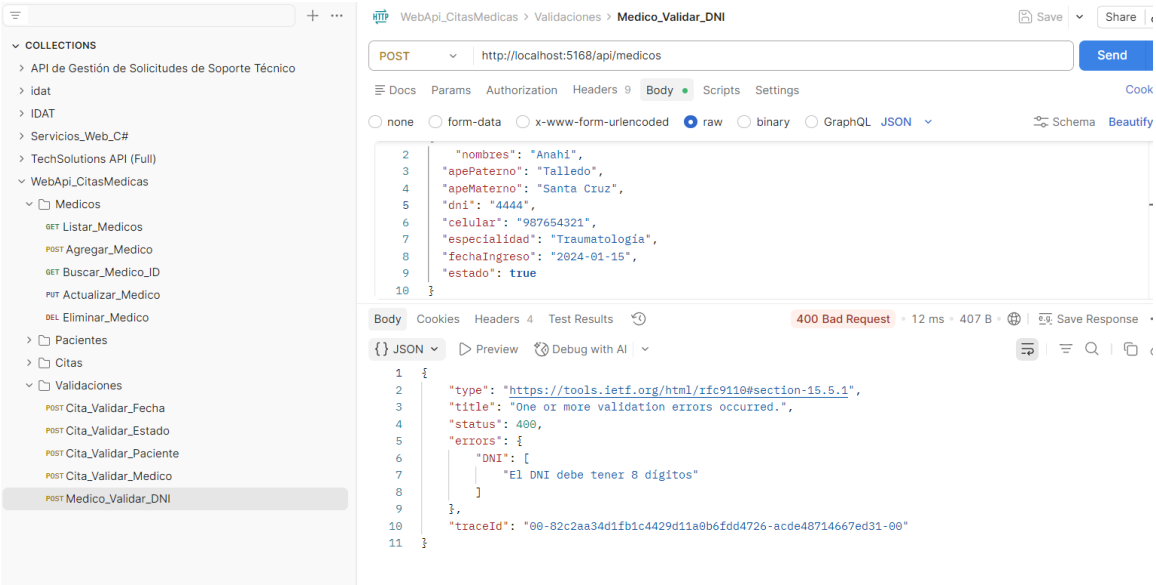
15.17. Prueba DELETE eliminar Médico /api/medicos/6

Eliminar_Medico: En esta prueba se verifica que el endpoint devuelve correctamente 200 OK, eliminando el médico con ID 6.



15.18. Validaciones POST Medico (Required / StringLength)

- **Medico_Validar_DNI:** En esta prueba validamos que, si enviamos un DNI que no tenga exactamente 8 dígitos, el endpoint nos devolverá el mensaje informativo correspondiente.



16. Conclusiones

El desarrollo de esta API permitió implementar un sistema básico de gestión de citas médicas
utilizando una arquitectura basada en API REST.

Durante el proyecto se aplicaron conceptos importantes como:

- Uso de controladores en ASP.NET Core
- Implementación de endpoints HTTP
- Uso de interfaces para desacoplar funcionalidades
- Validación de datos de entrada
- Pruebas de funcionamiento mediante Postman

El proyecto demuestra los fundamentos necesarios para el desarrollo de APIs modernas.